SOAR: filosofia i implementació d'una arquitectura cognitiva

De la teoria unificada de la cognició als agents basats en models generatius

---

Ramon Sangüesa

2026

Arquitectures cognitives per a la IA contemporània

Objectius de la presentació

1. Situar les **arquitectures cognitives** dins la història i el programa científic de la IA.
2. Explicitar els **principis filosòfics** que orienten SOAR.
3. Entendre com aquests principis es converteixen en una **maquinària computacional**.
4. Seguir un cicle complet: percepció, proposta, decisió, impasse, subestat i aprenentatge.
5. Analitzar què aporta SOAR als **agents basats en LLM** i què poden aportar els LLM a SOAR.

Pregunta central

Quina organització interna necessita un agent per actuar de manera coherent, aprendre de l'experiència i reutilitzar coneixement en tasques diferents?

Mapa de la presentació

1. El programa de les arquitectures cognitives
2. La filosofia de SOAR
3. Implementació computacional
4. Fortaleses, límits i comparació
5. SOAR davant els LLM i els agents generatius
6. Laboratori: conceptes de SOAR en CLIPS
7. Síntesi

Què és una arquitectura cognitiva?

Definició operativa

Una arquitectura cognitiva especifica les **estructures persistents** i els **processos invariants** que fan possible una família àmplia de conductes intel · ligents.

L'arquitectura fixa

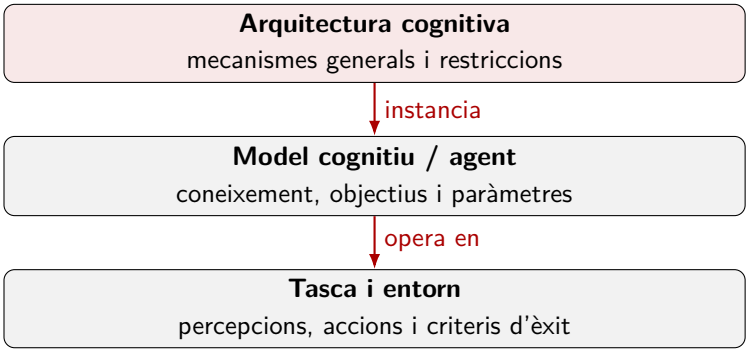
- tipus de memòria;
- representacions disponibles;
- mecanismes de decisió;
- formes d'aprenentatge;
- connexió amb percepció i acció.

El model o agent aporta

- coneixement d'un domini;
- objectius i preferències;
- regles i conceptes apresos;
- interfícies amb un entorn concret.

Referència conceptual: Newell (1990); Laird (2012).

Arquitectura, model cognitiu i aplicació



- Una arquitectura no és una teoria completa de cada tasca.
- Un mateix nucli arquitectònic pot sostenir molts agents.
- La restricció és científica: obliga a explicar fenòmens diferents amb mecanismes comuns.

Per què les arquitectures cognitives són IA fonamental?

- La IA no estudia només funcions d'entrada i sortida. També estudia **agents persistents** que decideixen al llarg del temps.
- Una arquitectura respon com es coordinen memòria, inferència, aprenentatge, objectius, percepció i acció.
- Proporciona una teoria executable: les hipòtesis han de produir conducta en un sistema real.
- Permet comparar explicacions pel que fa a generalitat, transferència, eficiència, robustesa i plausibilitat cognitiva.
- Fa visible què és coneixement de la tasca i què és mecanisme general.

Tesi

Sense arquitectura, les capacitats poden coexistir com una col · lecció de mòduls. Amb arquitectura, han de formar un sistema cognitiu integrat.

Un llinatge intel · lectual

Tradició	Idea que arriba a SOAR
Simon i Newell	La intel · ligència com a cerca heurística en espais de problemes.
Sistema físic de símbols	Els símbols i les estructures simbòliques poden sostenir acció intel · ligent general.
Nivell de coneixement	Cal explicar la conducta segons el coneixement que l'agent té i els seus objectius.
Sistemes de producció	El coneixement procedimental adopta la forma de regles condició–acció.
Teories unificades	Una sola arquitectura ha d'explicar moltes capacitats i tasques, no un mecanisme diferent per a cada fenomen.

Newell i Simon (1976); Newell (1982, 1990); Laird, Newell i Rosenbloom (1987).

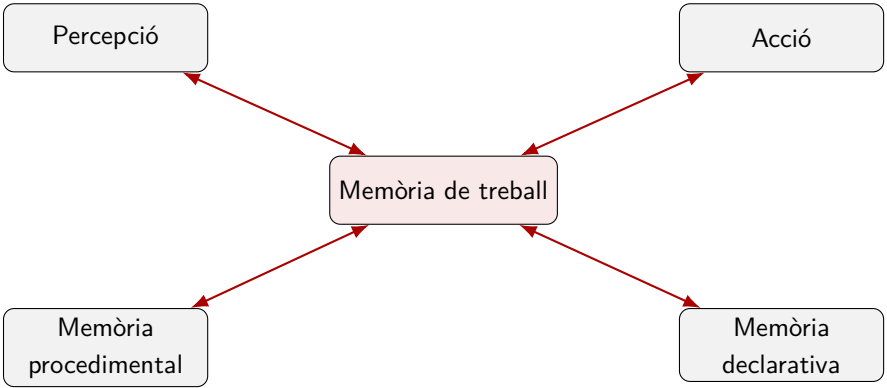
Famílies d'arquitectures cognitives

Orientació	Exemples	Èmfasi
Simbòlica	SOAR, ACT-R	Regles, estructures explícites, resolució de problemes i explicació.
Emergent / neural	Leabra, SPAUN	Dinàmiques distribuïdes, aprenentatge i restriccions neurocognitives.
Híbrida	CLARION, LIDA, Sigma	Integració simbòlica, sub-simbòlica, probabilística o perceptiva.
Agents cognitius	ICARUS, MIDCA i extensions modernes	Planificació, metacognició, autonomia situada i interacció.

SOAR dins el mapa

SOAR parteix d'un nucli simbòlic, però ha incorporat reforç, memòria semàntica, memòria episòdica, percepció espacial i coneixement numèric.

El Model Comú de Cognició



- Diverses arquitectures convergeixen en una organització de mòduls especialitzats coordinats per una memòria de treball limitada.
- SOAR és una de les fonts principals d'aquest consens i una de les seves realitzacions més completes.

Laird, Lebiere i Rosenbloom (2017), «A Standard Model of the Mind».

SOAR en una frase

Idea nuclear

La cognició és un cicle continu en què el coneixement proposa i avalua **operadors**; quan el coneixement no basta, el sistema crea deliberadament un **subestat**; quan resol la dificultat, **aprèn** a evitar-la en el futur.

situació $\longrightarrow$ operadors $\longrightarrow$ decisió $\longrightarrow$ acció / impasse $\longrightarrow$ aprenentatge

SOAR és alhora:

- una hipòtesi sobre l'organització de la cognició;
- un llenguatge per construir agents;
- un sistema de programari executable;
- un programa de recerca acumulatiu iniciat als anys vuitanta.

Principi 1: cognició general i integrada

- La intel · ligència general no s'obté apilant solucions independents per a cada tasca.
- Les mateixes estructures han de donar suport a raonament, planificació, aprenentatge, execució reactiva i recuperació de records.
- L'arquitectura ha de funcionar en tasques breus i en agents que persisteixen durant molt temps.
- El test fort és la **reutilització**: una capacitat nova ha d'encaixar amb les ja existents.

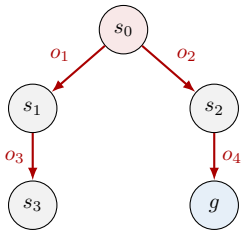
Conseqüència de disseny

Tots els mòduls aboquen els resultats rellevants a una memòria de treball comuna i el mecanisme de decisió manté la coherència de l'acció.

Principi 2: la conducta s'organitza en espais de problemes

Un espai de problemes conté:

- un **estat** que descriu la situació actual;
- un conjunt d'**operadors** possibles;
- coneixement per proposar-los i comparar-los;
- canvis d'estat produïts en aplicar-los;
- proves o indicis que indiquen progrés i finalització.



La noció continua el programa del General Problem Solver, però SOAR converteix l'operador en la unitat universal de decisió.

Principi 3: l'operador és la unitat de conducta

1. El coneixement disponible **elabora** la situació.
2. Algunes regles **proposen** operadors pertinents.
3. Altres regles expressen **preferències** entre operadors.
4. El procediment de decisió selecciona un operador.
5. Les regles d'aplicació produeixen canvis interns o accions externes.

Per què operadors?

La mateixa abstracció representa una acció motora, un pas de càlcul, una pregunta a la memòria, l'elecció d'un pla o una acció comunicativa.

Compromís filosòfic

Pensar i actuar són formes d'aplicar transformacions controlades sobre una situació representada.

Principi 4: tot el coneixement procedimental és uniforme

- Les produccions tenen la forma **si condicions, aleshores accions**.
- El mateix format expressa elaboració, proposta, avaluació i aplicació d'operadors.
- Les regles coincideixen en paral · lel amb la memòria de treball.
- El coneixement no queda lligat a un flux de control escrit manualment.

Separació clau

Coneixement: què és aplicable o preferible en aquesta situació.

Arquitectura: com es combinen les coincidències i com es pren la decisió.

Principi 5: el coneixement guia, l'arquitectura decideix

	Coneixement de l'agent	Mecanisme arquitectònic
Què pot fer?	Propostes d'operador	Detecta els candidats.
Què convé?	Preferències i valors	Resol la selecció.
Com s'executa?	Regles d'aplicació	Manté el cicle.
Què passa si no sap?	Coneixement parcial	Declara un impasse.
Com millora?	Resultats del raonament	Compila un nou *chunk*.

El control emergeix de la interacció entre coneixement local i processos generals. No cal un planificador central separat que conegui totes les tasques.

Principi 6: els impasses fan explícits els límits del coneixement

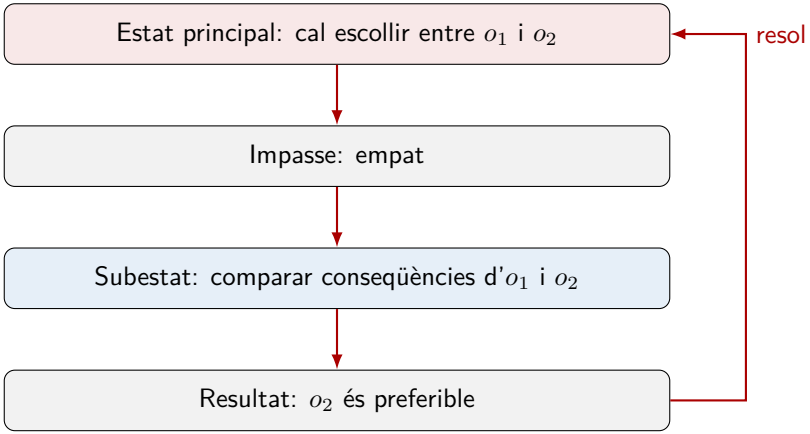
Un **impasse** apareix quan el procediment de decisió no pot avançar:

- no hi ha cap operador proposat;
- hi ha diversos candidats sense preferència suficient;
- les preferències entren en conflicte;
- l'operador seleccionat no pot aplicar-se o no produeix progrés.

Interpretació

L'impasse no és un error fatal. És una declaració arquitectònica: *el coneixement immediat no determina què cal fer.*

Principi 7: subestats per raonar sobre el problema de control



- El subestat és un context de resolució de problemes creat automàticament.
- Pot contenir altres operadors i generar impasses niats.
- La jerarquia de subestats emergeix de les necessitats del raonament.

Principi 8: una estratègia universal de reserva

- Si l'agent té coneixement específic, actua directament.
- Si el coneixement és insuficient, l'impasse desencadena cerca, simulació, recuperació o anàlisi en un subestat.
- Qualsevol **mètode feble** pot emergir del coneixement disponible: cerca en profunditat, mitjans–fins, ascens de turó o planificació.
- Per això es parlava d'un **mètode feble universal**: l'arquitectura no imposa una sola heurística, però sempre ofereix una manera de continuar.

Idea de fons

La flexibilitat no prové d'un algorisme universal omniscient, sinó de poder convertir qualsevol dificultat de decisió en un nou problema representable.

Principi 9: aprendre vol dir convertir deliberació en reacció

Primera vegada

1. apareix un impasse;
2. es crea un subestat;
3. l'agent raona;
4. obté un resultat.

Vegades següents

1. una nova producció reconeix les condicions rellevants;
2. produeix directament el resultat;
3. evita repetir tota la deliberació.

Chunking

El *chunking* explica el resultat del subestat a partir de les condicions que el van fer necessari i crea una regla reutilitzable.

Principi 10: racionalitat limitada i cognició situada

- L'agent només decideix a partir del contingut actual de la memòria de treball i del coneixement recuperable.
- La conducta depèn del temps, de les percepcions disponibles i de la història de l'agent.
- No s'assumeix una optimització global perfecta.
- Les preferències poden ser parcials, qualitatives o numèriques.
- Aprendre canvia el cost de la cognició futura: una deliberació llarga pot esdevenir una resposta immediata.

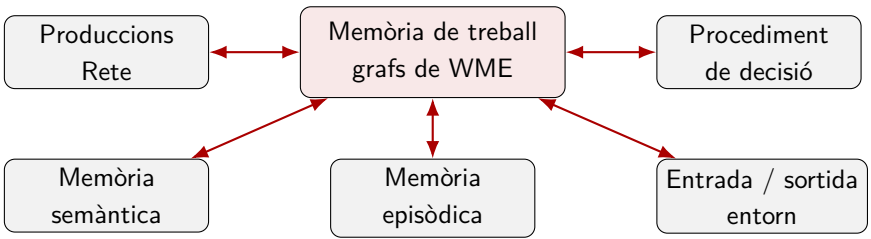
Lectura cognitiva

La competència observable depèn tant del coneixement com de la manera arquitectònica d'accedir-hi i utilitzar-lo.

Principi 11: modularitat de memòries, integració de conducta

Memòria	Contingut	Funció
Treball	Situació activa i context	Espai compartit de coordinació.
Procedimental	Produccions	Saber com actuar i deliberar.
Semàntica	Fets i conceptes descontextualitzats	Saber què és cert o típic.
Episòdica	Instantànies temporals de l'experiència	Recordar què va passar i en quin ordre.
Reforç	Valors associats a decisions	Ajustar preferències segons recompensa.
Espacial-visual	Escenes, objectes i relacions espacials	Connectar representació simbòlica i entorn percebut.

Vista general de la implementació



El nucli executa cicles de coincidència de produccions i fases de decisió. Els mòduls especialitzats intercanvien estructures simbòliques a través de la memòria de treball.

La memòria de treball com a graf simbòlic

Una WME és una tripleta:

$$(\textit{identificador}\ \ \hat{}\textit{atribut}\ \ \textit{valor})$$

Exemple de WME

```
(S1 ^name robot-room
    ^battery 42
    ^operator O1
    ^sensor L1)
(L1 ^kind lidar
    ^obstacle true)
(O1 ^name turn-left)
```

- Els identificadors connecten objectes.
- Els atributs creen arcs etiquetats.
- Els valors poden ser constants o identificadors.
- L'estructura resultant és un graf dirigit.

Important

La memòria de treball conté el context actiu, no tot el coneixement de l'agent.

Produccions: condicions i accions

Regla esquemàtica de proposta

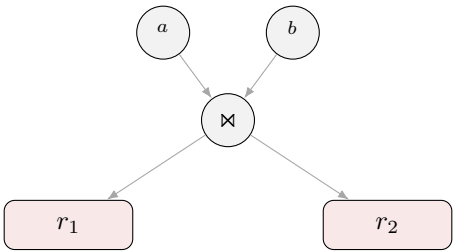
```
sp {propose*turn-left
   (state <s> ^sensor <l>)
   (<l> ^obstacle true)
-->
   (<s> ^operator <o> +)
   (<o> ^name turn-left)
}
```

- La part esquerra cerca un patró en memòria de treball.
- La part dreta crea o elimina WME, o bé produeix preferències.
- Les variables uneixen estructures distribuïdes pel graf.
- Moltes instanciacions poden disparar-se en una mateixa elaboració.

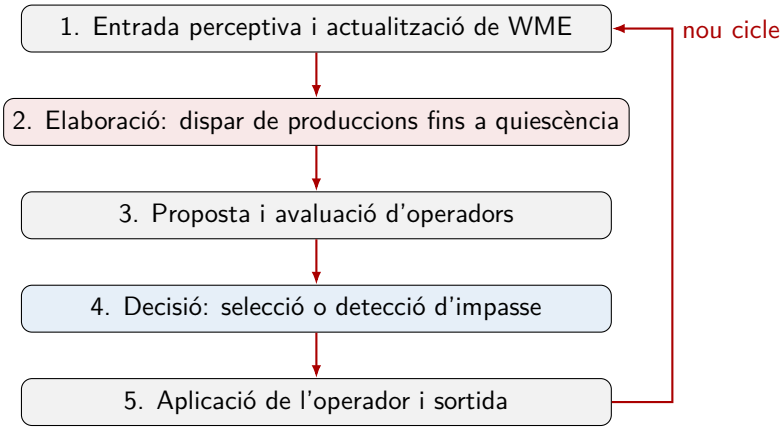
La sintaxi exacta pot variar segons la versió i les convencions del model; l'exemple mostra el patró conceptual habitual.

Rete: coincidència incremental eficient

- Comprovar totes les regles des de zero a cada canvi seria costós.
- La xarxa **Rete** comparteix proves comunes entre regles.
- Conserva coincidències parcials i actualitza només les parts afectades per WME noves o eliminades.
- El cost depèn sobretot dels canvis, no de rellegir sempre tota la memòria.



El cicle de decisió



Les descripcions pedagògiques agrupen o separen fases de manera lleugerament diferent; el principi és l'alternança entre elaboració paral · lela i decisió serial.

Elaboració paral · lela, decisió serial

Elaboració

- moltes produccions coincideixen;
- s'afegeixen inferències compatibles;
- no hi ha una seqüència manual de regles;
- continua fins que no apareixen canvis rellevants.

Decisió

- integra les preferències;
- selecciona un únic operador per estat;
- crea un impasse si la selecció és indeterminada;
- proporciona continuïtat temporal a la conducta.

Arquitectura híbrida de control

SOAR combina inferència associativa massiva amb un coll d'ampolla deliberatiu que manté una línia d'acció coherent.

Preferències: més que una puntuació

Les produccions poden afirmar relacions sobre els operadors:

- **acceptable**: l'operador és candidat;
- **reject**: queda exclòs;
- **better / worse**: ordenació relativa;
- **best / worst**: qualificació extrema;
- **require / prohibit**: restricció forta;
- **indifferent**: no hi ha base qualitativa per distingir;
- **numeric indifferent**: valor numèric utilitzable, també per aprenentatge per reforç.

Conseqüència

La decisió pot combinar coneixement simbòlic qualitatiu, restriccions i valors apresos sense reduir-ho tot a una sola funció d'utilitat.

Tipus d'impasse i subestat generat

Impasse	Situació	Problema que el subestat pot resoldre
No-change	Cap canvi permet avançar	Descobrir com aplicar l'operador o transformar l'estat.
Tie	Diversos operadors empaten	Comparar candidats, simular conseqüències o recuperar experiència.
Conflict	Preferències incompatibles	Revisar criteris o obtenir coneixement discriminant.
Constraint failure	Requisits impossibles de satisfer	Diagnosticar supòsits o reformular el problema.

Els subestats hereten informació pertinent de l'estat superior, però mantenen un context propi per al raonament metanivell.

Chunking: explicació operacional de l'aprenentatge

1. El subestat produeix un **resultat** que modifica l'estat superior.
2. SOAR rastreja quines produccions i quines WME van contribuir al resultat.
3. Identifica les condicions de l'estat superior de les quals depenia el raonament.
4. Construeix una nova producció: condicions rellevants $\Rightarrow$ resultat.
5. La regla entra a la memòria procedimental i pot disparar-se en situacions futures.

Efecte funcional

Composició de regles, generalització limitada per variables i transferència del resultat fora del context concret del subestat.

El problema del chunking excessiu

- Un agent pot crear massa regles, massa específiques o amb condicions accidentals.
- Una regla apresa incorrectament pot disparar-se en contextos inadequats.
- L'augment de produccions pot empitjorar el rendiment de coincidència.
- Els models reals necessiten disseny acurat de representacions, control de dependències i eines de depuració.

Lliçó general

Aprendre una drecera és útil només si l'explicació causal o operacional que la justifica conserva les condicions pertinents.

Memòria semàntica

- Emmagatzema estructures declaratives relativament estables: conceptes, fets i associacions.
- Les recuperacions parteixen d'una consulta especificada en memòria de treball.
- El resultat recuperat torna a memòria de treball i pot activar produccions.
- L'activació i altres mecanismes permeten seleccionar entre records candidats.

Exemple

Consulta: «quin tipus d'objecte és una tassa i com sol manipular-se?»

Resposta: propietats conceptuals que després condicionen els operadors disponibles.

Memòria episòdica

- Registra seqüències d'instantànies de la memòria de treball.
- Permet consultar episodis semblants a una configuració parcial.
- Conserva ordre temporal i context, a diferència de la memòria semàntica.
- Pot donar suport a predicció, explicació, recuperació d'un pla anterior o resposta a preguntes sobre l'experiència.

Diferència funcional

Semàntica: «què sé?»

Episòdica: «quan vaig trobar una situació semblant i què va passar després?»

Aprenentatge per reforç

- Les regles de preferència numèrica poden associar valors als operadors.
- La recompensa actualitza aquests valors amb mecanismes temporals de reforç.
- El procediment de decisió utilitza els valors dins el mateix marc de preferències.
- El reforç no substitueix el coneixement simbòlic. Ajusta eleccions dins una estructura explícita de tasca.

$$Q(s,o) \leftarrow Q(s,o) + \alpha\big[r + \gamma Q(s',o') - Q(s,o)\big]$$

L'equació mostra la idea general d'una actualització temporal; els detalls depenen de la configuració del mòdul de RL de SOAR.

Percepció, acció i entorn

- Els enllaços d'entrada tradueixen l'estat de l'entorn a WME.
- Els enllaços de sortida converteixen comandes simbòliques en accions del simulador, robot o aplicació.
- El *Spatial Visual System* pot mantenir representacions geomètriques més riques que el graf simbòlic ordinari.
- La interfície determina quines regularitats del món són accessibles a l'agent.

Punt metodològic

La representació perceptiva no és una simple canonada tècnica: defineix part del problema cognitiu que l'agent pot formular.

Exemple complet: robot davant d'un obstacle

Pas	Estat de la computació
1	El lidar introdueix `obstacle=true`; la bateria i l'objectiu ja són a memòria de treball.
2	Les produccions proposen `turn-left`, `turn-right` i `stop`.
3	El coneixement rebutja `stop` si hi ha una ruta segura i prefereix girar cap al costat amb més espai.
4	Si els dos costats empaten, el procediment de decisió crea un impasse *tie*.
5	En el subestat, l'agent consulta el mapa semàntic o recupera un episodi semblant.
6	El resultat prefereix `turn-left`; el nivell superior selecciona i aplica l'operador.
7	El *chunking* crea una regla que resol directament futurs empats amb les mateixes condicions rellevants.

Traça conceptual d'execució

Traça simplificada

```
INPUT: obstacle=true, left-clear=0.8, right-clear=0.8
PROPOSE: turn-left, turn-right
PREFERENCES: indifferent(turn-left, turn-right)
DECISION: tie impasse
  SUBSTATE: retrieve similar episode
  EPMEM: previous turn-left -> goal preserved
  RESULT: better(turn-left, turn-right)
DECISION: select turn-left
APPLY: output-link command=turn-left
LEARN: chunk resolves future matching ties
```

Què cal observar

La traça mostra una continuïtat entre conducta reactiva, deliberació, memòria autobiogràfica i compilació de coneixement.

Enginyeria d'un agent SOAR

1. Definir el vocabulari de la memòria de treball i les interfícies d'entrada/sortida.
2. Identificar estats, operadors i resultats esperats.
3. Escriure regles d'elaboració, proposta, preferència i aplicació.
4. Decidir què pertany a memòria procedimental, semàntica o episòdica.
5. Executar tasques i inspeccionar traces, preferències i impasses.
6. Refinar representacions i validar les regles apreses.
7. Mesurar èxit, temps de decisió, transferència i creixement del coneixement.

La dificultat principal

El repte no sol ser escriure una regla aïllada, sinó construir representacions que permetin reutilització i aprenentatge correcte.

Fortaleses de SOAR

- **Integració**: decisió, múltiples memòries i diverses formes d'aprenentatge dins un únic cicle.
- **Transparència**: estats, operadors, preferències i regles poden inspeccionar-se.
- **Aprenentatge explicable**: el *chunking* vincula una regla nova amb el raonament que la va produir.
- **Autonomia temporal**: adequat per a agents que perceben, actuen i persisteixen.
- **Acumulació**: dècades de models, eines, aplicacions i refinament teòric.
- **Disciplina científica**: força a concretar com interactuen capacitats que sovint s'estudien separadament.

Límits i tensions

- Cost elevat de modelatge simbòlic i d'enginyeria del coneixement.
- Sensibilitat a la representació escollida per estats, operadors i atributs.
- Percepció i llenguatge oberts resulten difícils sense components externs.
- El *chunking* pot generar regles fràgils o una explosió de coneixement.
- La plausibilitat cognitiva d'alguns mecanismes és discutible o menys detallada que en arquitectures centrades en predicció psicològica.
- La integració de mecanismes simbòlics, estadístics i neuronals continua sent una línia oberta.

Valor del límit

Aquestes tensions identifiquen on una teoria d'agent necessita percepció apresa, representacions més flexibles i control metacognitiu.

SOAR i ACT-R: semblances i diferències

Dimensió	SOAR	ACT-R
Objectiu històric	Intel · ligència general integrada i resolució de problemes.	Modelatge cognitiu humà i predicció experimental.
Control	Operadors, preferències, impasses i subestats.	Selecció de produccions amb utilitats i buffers modulars.
Memòria activa	Graf simbòlic relativament ric.	Buffers limitats que connecten mòduls.
Aprenentatge distintiu	*Chunking* a partir de subestats, més RL.	Compilació de produccions, utilitats i aprenentatge declaratiu.
Temporalitat	Cicle de decisió orientat a agent.	Temps cognitiu quantitatiu i latències de mòduls.
Punt comú	Produccions, memòria de treball, memòria declarativa i cognició integrada.	

Laird (2022), «An Analysis and Comparison of ACT-R and Soar».

Per què aquesta discussió torna a ser urgent?

Els models generatius han ampliat extraordinàriament:

- l'accés flexible a coneixement lingüístic i cultural;
- la generalització per instruccions i exemples;
- la generació de plans, codi i explicacions;
- la interfície entre humans i sistemes computacionals.

Però un agent persistent encara necessita resoldre:

- identitat, objectius i estat al llarg del temps;
- selecció d'accions i arbitratge entre alternatives;
- memòria episòdica fiable i aprenentatge acumulatiu;
- verificació, recuperació d'errors i control dels recursos.

Model fundacional i arquitectura cognitiva

Pregunta	Model generatiu	Arquitectura cognitiva
Què aprèn?	Regularitats d'un gran corpus i ajustos posteriors.	Coneixement d'una vida d'agent dins mecanismes definits.
Què persisteix?	Pesos i context proporcionat a cada crida.	Memòries, objectius, estat i habilitats adquirides.
Com decideix?	Genera una continuació condicionada.	Avalua alternatives dins un cicle d'acció.
Com explica?	Genera una narració plausible.	Pot exposar regles, preferències, impasses i traces causals.
Com actua?	Necessita una bastida externa d'eines i control.	Inclou explícitament el bucle percepció–decisió–acció.

No són categories excloents

Un model generatiu pot esdevenir un component potent dins una arquitectura d'agent.

Què ofereix SOAR als agents basats en LLM?

- Una representació explícita de l'**estat operatiu** de l'agent.
- Un mecanisme estable per proposar, comparar i seleccionar **accions**.
- Detecció estructural de situacions on falta coneixement: els **impasses**.
- Separació entre memòria activa, semàntica, episòdica i procedimental.
- Una via per convertir solucions deliberades en habilitats reutilitzables.
- Traces que permeten auditar per què l'agent va consultar un model, una eina o una memòria.

Idea útil

SOAR pot actuar com a sistema executiu i el LLM com a font de coneixement lingüístic, hipòtesis o operadors candidats.

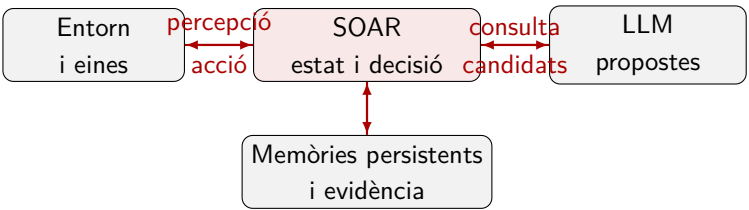
Què ofereixen els LLM a una arquitectura com SOAR?

- Traducció entre llenguatge natural i representacions simbòliques.
- Adquisició ràpida de coneixement de domini que abans exigia codificació manual.
- Generació d'operadors candidats en situacions noves.
- Resums de memòria episòdica i creació d'abstraccions semàntiques.
- Interacció conversacional i explicacions adaptades a l'usuari.
- Percepció multimodal quan el model integra text, imatge i àudio.

Condició

Les propostes generatives necessiten validació, ancoratge en l'estat real i límits d'autoritat abans de convertir-se en accions.

Patró híbrid: SOAR com a executiu, LLM com a recurs



1. Un impasse pot activar una consulta al LLM.
2. El LLM proposa opcions, no executa directament.
3. Regles, verificadors o eines comproven precondicions i conseqüències.
4. El procediment de decisió integra les preferències i manté la responsabilitat de control.

Riscos del patró híbrid

- **Desalineació representacional**: el text generat no coincideix amb l'estat simbòlic real.
- **Autoritat excessiva**: una proposta lingüísticament convincent salta les restriccions de decisió.
- **Memòria contaminada**: contingut no verificat entra a memòria semàntica o episòdica.
- **Aprenentatge prematur**: el sistema compila una resposta accidental en una regla persistent.
- **Cost i latència**: consultar el model en cada cicle impedeix conducta reactiva.
- **Explicació confosa**: la justificació generada es presenta com si fos la traça causal real.

Principi de seguretat

Distingir sempre proposta, evidència, decisió, acció i explicació causal.

SOAR i CLIPS: proximitat i diferència

Aspecte	SOAR	CLIPS
Nucli compartit	Sistemes de producció amb coincidència eficient mitjançant Rete.	
Control	Cicle de decisió arquitectònic, operadors, preferències i impasses.	Agenda de regles, saliència, estratègies de resolució de conflictes i control programat.
Memòria activa	Graf de WME organitzat al voltant d'estats.	Fets ordenats o plantilles `deftemplate`; també objectes.
Aprenentatge	*Chunking*, reforç, memòria semàntica i episòdica.	No implementa directament el *chunking* de SOAR ni les seves memòries arquitectòniques.
Finalitat	Arquitectura cognitiva general.	Llenguatge i entorn per construir sistemes basats en regles.

Precaució didàctica

CLIPS permet reconstruir i entendre alguns principis de SOAR, però una simulació en CLIPS no es converteix per això en un agent SOAR.

Correspondència didàctica SOAR–CLIPS

Concepte SOAR	Construcció CLIPS	Què cal afegir explícitament
WME	`deftemplate` i fets	Identificadors i enllaços entre fets.
Producció	`defrule`	Convencions per distingir elaboració, proposta i aplicació.
Operador candidat	Fet `operator`	Identitat, estat, fase i justificació.
Preferència	Fet `preference`	Regles d'arbitratge i tractament de conflictes.
Decisió	Regles de selecció	Garantir una sola selecció per cicle.
Impasse	Fet diagnòstic	Detecció de no-candidat, empat, conflicte o no-canvi.
Subestat	Fet `state` amb pare	Pila o jerarquia de contextos.
Chunking	Regla generada o meta-programació	Anàlisi de dependències i validació de la regla nova.

CLIPS: representar estat, operadors i preferències

```
(deftemplate state
  (slot id) (slot phase) (slot obstacle)
  (slot left-clear) (slot right-clear))

(deftemplate operator
  (slot id) (slot state) (slot name) (slot status))

(deftemplate preference
  (slot state) (slot better) (slot worse) (slot reason))

(defrule propose-turn-left
  (state (id ?s) (phase propose) (obstacle yes))
  (not (operator (state ?s) (name turn-left)))
  =>
  (assert (operator (id (gensym*)) (state ?s)
                    (name turn-left) (status candidate))))
```

Lectura SOAR

La regla només proposa un candidat. No barreja proposta, selecció i execució en una única producció.

CLIPS: preferència, decisió i aplicació

```
(defrule prefer-more-clearance
  (state (id ?s) (phase decide)
         (left-clear ?l) (right-clear ?r&:(> ?r ?l)))
  (operator (id ?ol) (state ?s) (name turn-left))
  (operator (id ?or) (state ?s) (name turn-right))
  =>
  (assert (preference (state ?s) (better ?or) (worse ?ol)
                      (reason clearance))))

(defrule select-better
  ?st <- (state (id ?s) (phase decide))
  ?op <- (operator (id ?best) (state ?s) (status candidate))
  (preference (state ?s) (better ?best))
  =>
  (modify ?op (status selected))
  (modify ?st (phase apply)))
```

Objectiu pedagògic

Separar les fases fa visible la diferència entre reconèixer una acció possible i comprometre's a executar-la.

CLIPS: detectar un empat com a impasse

```
(deftemplate impasse
  (slot state) (slot type) (multislot operators))

(defrule detect-tie
  ?st <- (state (id ?s) (phase decide)
                (left-clear ?x) (right-clear ?x))
  (operator (id ?ol) (state ?s) (name turn-left)
            (status candidate))
  (operator (id ?or) (state ?s) (name turn-right)
            (status candidate))
  (not (preference (state ?s) (better ?)))
  =>
  (assert (impasse (state ?s) (type tie)
                   (operators ?ol ?or)))
  (modify ?st (phase substate)))
```

Exercici

Crear un subestat que consulti un episodi anterior, generi una preferència i retorni el resultat a l'estat pare.

Pràctica de 90 minuts amb CLIPS

1. **Representació** (15 min): crear plantilles per a estat, operador, preferència i impasse.
2. **Proposta** (15 min): generar tres operadors sense seleccionar-los encara.
3. **Avaluació** (15 min): introduir preferències qualitatives i una restricció forta.
4. **Decisió** (15 min): implementar una selecció única i registrar-ne la justificació.
5. **Impasse** (15 min): detectar un empat i obrir un context subordinat.
6. **Reflexió** (15 min): identificar què ha proporcionat CLIPS i què forma part de l'arquitectura SOAR que hem hagut de programar.

Extensió

Afegir un simulador petit en Python, separar entrada i sortida, i comparar la traça CLIPS amb la d'un agent SOAR equivalent.

Una agenda de recerca contemporània

- Aprendre representacions simbòliques a partir de percepció i llenguatge.
- Fer que els impasses dirigeixin consultes generatives selectives i econòmiques.
- Validar operadors generats mitjançant simulació, eines formals o entorns segurs.
- Consolidar episodis en coneixement semàntic sense introduir falsedats.
- Desenvolupar metacognició sobre capacitats, incertesa i límits del mateix agent.
- Comparar agents híbrids en transferència, robustesa, traçabilitat i aprenentatge continu.

Pregunta oberta

Podem combinar la flexibilitat estadística dels models fundacionals amb una arquitectura explícita de memòria, decisió i aprenentatge acumulatiu?

Onze principis en una sola vista

1. La cognició general exigeix integració arquitectònica.
2. La conducta s'organitza en estats i espais de problemes.
3. L'operador és la unitat universal de decisió i conducta.
4. El coneixement procedimental adopta un format uniforme de produccions.
5. El coneixement proposa i avalua; l'arquitectura arbitra.
6. Els impasses revelen una insuficiència de coneixement.
7. Els subestats converteixen problemes de control en problemes resolubles.
8. Els mètodes febles apareixen com a estratègia general de reserva.
9. El *chunking* transforma deliberació en resposta directa.
10. La cognició és limitada, temporal i situada.
11. Memòries especialitzades contribueixen a una conducta unificada.

Quatre idees que convé retenir

1. **Una arquitectura cognitiva és una teoria executable de l'agent**, no una simple biblioteca de funcions.
2. **SOAR situa la decisió al centre**: el coneixement produeix preferències sobre operadors i l'arquitectura resol la selecció.
3. **La manca de coneixement impulsa la cognició**: els impasses creen subestats i el *chunking* conserva les solucions.
4. **Els LLM reactualitzen el problema arquitectònic**: aporten coneixement flexible, però necessiten estat, memòria, control i verificació per esdevenir agents persistents.

Preguntes per a discussió

- L'operador continua sent una bona unitat de conducta en agents que actuen amb llenguatge?
- Un impasse pot servir com a política general per decidir quan consultar un LLM?
- Quina part d'una traça híbrida constitueix una explicació genuïna?
- Com podem impedir que una memòria persistent consolidi una al · lucinació?
- El *chunking* i la destil · lació de raonaments generatius són fenòmens comparables?
- Quines proves distingirien una arquitectura integrada d'una bastida de mòduls?

Bibliografia essencial I: fonaments


Laird, J. E. (2012). *The Soar Cognitive Architecture*. MIT Press.


Laird, J. E., Newell, A. i Rosenbloom, P. S. (1987). «SOAR: An Architecture for General Intelligence». *Artificial Intelligence*, 33(1), 1–64.


Newell, A. (1990). *Unified Theories of Cognition*. Harvard University Press.


Laird, J. E., Lebiere, C. i Rosenbloom, P. S. (2017). «A Standard Model of the Mind». *AI Magazine*, 38(4), 13–26.


Laird, J. E. (2022). «An Analysis and Comparison of ACT-R and Soar». arXiv:2201.09305.


Newell, A. (1982). «The Knowledge Level». *Artificial Intelligence*, 18(1), 87–127.


Newell, A. i Simon, H. A. (1976). «Computer Science as Empirical Inquiry: Symbols and Search». *Communications of the ACM*, 19(3), 113–126.


Rosenbloom, P. S., Laird, J. E. i Newell, A. (eds.) (1993). *The Soar Papers*. MIT Press.

Bibliografia essencial II: memòria, aprenentatge i integració


Nuxoll, A. M. i Laird, J. E. (2007). «Extending Cognitive Architecture with Episodic Memory». *AAAI*.


Derbinsky, N. i Laird, J. E. (2009). «Efficiently Implementing Episodic Memory». *ICCM*.


Nason, S. i Laird, J. E. (2005). «Soar-RL: Integrating Reinforcement Learning with Soar». *Cognitive Systems Research*, 6(1), 51–59.


Laird, J. E., Rosenbloom, P. S. i Newell, A. (1986). «Chunking in Soar: The Anatomy of a General Learning Mechanism». *Machine Learning*, 1, 11–46.


Laird, J. E. et al. (2025). «A Proposal to Extend the Common Model of Cognition with Metacognition». arXiv:2506.07807.


Kotseruba, I. i Tsotsos, J. K. (2020). «40 Years of Cognitive Architectures: Core Cognitive Abilities and Practical Applications». *Artificial Intelligence Review*, 53, 17–94.

Programari, manuals i repositoris

- **Portal oficial de SOAR**: manuals, tutorials i descàrregues.
  `https://soar.eecs.umich.edu/`
- **Nucli oficial de SOAR**: codi font, compilació, proves i interfícies SML.
  `https://github.com/SoarGroup/Soar`
- **Materials de distribució i exemples**: repositoris de l'organització SoarGroup.
  `https://github.com/SoarGroup`
- **CLIPS oficial**: descàrregues, guies 6.4.2, suport i exemples.
  `https://www.clipsrules.net/`
- **CLIPS a SourceForge**: codi, binaris i paquets d'exemples.
  `https://sourceforge.net/projects/clipsrules/`
- **Repositoris de Gary Riley**: implementacions i exemples mantinguts pel desenvolupador principal de CLIPS.
  `https://github.com/garydriley`

Enllaços comprovats el setembre de 2026. El repositori oficial de SOAR declara suport multiplataforma i interfícies SWIG per a Python, Java, JavaScript, C#, Tcl i altres entorns.

SOAR proposa que una ment artificial general no és només un model que produeix respostes, sinó una arquitectura que manté una vida cognitiva.